推理服务挑战赛 Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: 搭建完整的竞赛推理服务系统，包含热身阶段本地模拟平台和正式预赛提交框架，以 SGLang 为推理后端，最大化比赛得分。

Architecture: 系统分两层：(1) Mock Platform 模拟官方评测服务器，供热身阶段本地调试；(2) Contestant Service 包含 SGLang 推理后端 + 调度客户端，通过读取 `CONFIG_PATH` 环境变量无缝切换热身/正式模式，提交时只需打包 contestant/ 目录。

Tech Stack: Python 3.12, FastAPI, SGLang（本地 clone + editable install，推理后端）, httpx（异步 HTTP 客户端）, pytest, uvicorn

---

基线选型：SGLang（推荐）

维度	SGLang	vLLM	nano-vllm
Prefix Caching	RadixAttention（token 级树结构，自动命中）	块级哈希，需手动开启	基础实现
吞吐量（H100）	~16,200 tok/s	~12,500 tok/s	未公开
logprob API	原生 `/generate?return_logprob=true`	echo+logprobs，较成熟	有限
可修改性	中等	复杂	高（1200 行）
生产就绪	是	是	否

选 SGLang 的核心原因：

1. 本赛题 loglikelihood 任务中，同一 prompt 对应多个 continuation（多选题），这正是 RadixAttention 的最强场景——所有 continuation 共享 prompt 的 KV Cache，减少重复计算。
2. 原生支持 `return_logprob + input_token_logprobs`，直接满足 loglikelihood 计算需求。
3. TTFT 更低，有助于命中 Supreme/Glorious 等高 SLA 收益。

热身阶段模型：Qwen2.5-7B-Instruct（同系列、行为最接近赛题 Qwen3-32B，单卡即可运行）

热身阶段 vs 正式预赛：差别预期

维度	热身阶段（本计划）	正式预赛
平台 URL	`http://localhost:8003`（本地 mock）	官方平台（CONFIG_PATH 注入）

维度	热身阶段（本计划）	正式预赛
模型	Qwen2.5-7B（本地自备）	Qwen3-32B（/mnt/model/ 固定路径）
硬件	开发机（任意 GPU）	官方统一环境（GPU 型号待定）
任务流	mock_platform 生成（可重放、可控）	官方随机，不可预测
代码	contestant/ 完整代码	相同代码，只改环境变量
提交方式	本地直接运行	打 .tar.gz 上传平台
得分	本地日志/数据库	官方排行榜
关键风险	无（本地调试）	run.sh 必须 60s 内启动；GPU 内存约束

核心设计原则：contestant/ 代码与热身/正式环境完全解耦，靠 CONFIG_PATH 注入切换，零代码修改。

SGLang 本地部署与修改策略

为什么要 clone 源码

比赛环境是一个封闭局域网 GPU 机器，`run.sh` 在这台机器上运行，SGLang 必须部署在**本地**（`localhost:30000`），不能调用外部服务。clone 源码并 editable install 有两个额外好处：

1. 可以直接修改 SGLang 内部逻辑（如调度器），改完立即生效，无需重新打包
2. 提交时把 `sglang/` 目录一起打包进 `.tar.gz`，保证环境一致

```
# 初次设置（在开发机上执行）
git clone https://github.com/sgl-project/sglang.git
cd sglang
pip install -e "python/[all]"   # editable install，修改源码即时生效
```

SGLang 核心修改点（按优先级）

以下是本赛题中值得修改的 SGLang 内部逻辑，**当前 Task 阶段不实施**，在热身系统跑通后、预赛前评估是否实施：

修改点 1（最高优先级）：Scheduler 优先级队列

文件： `sglang/python/sglang/srt/managers/scheduler.py`

问题： SGLang 默认使用 FIFO 队列（`waiting_queue: deque`）。本赛题中 Supreme SLA（ttft=0.5s）的任务等待时间必须极短，而 Bronze（ttft=10s）的任务可以等。FIFO 无法区分。

修改方案： 将 `waiting_queue` 改为优先级堆，按 SLA 等级排序。

```
# scheduler.py 原始（约第 200 行附近）
self.waiting_queue: List[Req] = []
```

```
# 修改后
import heapq
self.waiting_queue: List[Req] = []   # 改为 heapq 维护

# 在 add_req() 中，将 append 改为 heappush
#  (需在 Req 上添加 priority 字段并实现 __lt__)
```

对应的 Req 修改： `sglang/python/sglang/srt/managers/io_struct.py`

```
# 在 Req.__init__ 中添加
self.priority: int = 0   # 0=lowest(Bronze), 7=highest(Supreme)

def __lt__(self, other: "Req") -> bool:
    return self.priority > other.priority  # 数值越大越优先
```

HTTP API 扩展： 在 `/generate` 接口接受 `priority` 参数，contestant 调用时传入 SLA 对应的 priority 值（Bronze=0 … Supreme=7）。

修改点 2（中优先级）：KV Cache 逐出策略

文件： `sglang/python/sglang/srt/mem_cache/radix_cache.py`

问题： 默认逐出策略是 LRU（最近最少使用）。对于本赛题，低 SLA 任务的 prefix 应该比高 SLA 任务的 prefix 优先被逐出。

修改方案： 在 RadixCache 的 `evict()` 方法中，按节点关联的最高 SLA priority 保留缓存，优先淘汰 Bronze 任务的缓存块。（实施复杂度较高，留到性能瓶颈出现后再考虑）

修改点 3（低优先级）：Chunked Prefill 批组合策略

文件： `sglang/python/sglang/srt/managers/scheduler.py` 的 `get_next_batch_to_run()`

问题： loglikelihood 任务（只需前向传播，无生成）和 generate_until 任务（需自回归生成）混跑时，批次组合影响吞吐量和延迟。

修改方案： 将纯 loglikelihood 请求优先组成独立批次（无需 decode 阶段），避免和长生成任务混跑拖慢 TTFT。

文件结构

```
quant/
├── sglang/                      # git clone 的 SGLang 源码（本地 editable
install)
│   └── python/sglang/srt/
│       ├── managers/scheduler.py     #  【待修改】优先级队列
```

```
│        ├── managers/io_struct.py     #  【待修改】Req.priority 字段
│        └── mem_cache/radix_cache.py  #  【可选修改】KV Cache 逐出策略
│
├── mock_platform/                     # 热身阶段：模拟官方评测服务器
│    ├── server.py                    # FastAPI 主服务（/register /query /ask
/submit）
│    ├── task_generator.py            # 任务流生成器（含重复 prompt、多选题拆分）
│    ├── scorer.py                    # 得分计算（含 SLA 判定、惩罚逻辑）
│    └── config.py                    # mock 配置（SLA 等级、采样参数定义）
│
├── contestant/                        # 正式提交代码（打包此目录）
│    ├── run.sh                       # 启动脚本（必须）
│    ├── setup.sh                     # 环境安装脚本
│    ├── requirements.txt
│    ├── main.py                      # 主入口：读 CONFIG_PATH，启动后端+客户端
│    ├── client.py                    # 平台 API 客户端
 (register/query/ask/submit)
│    ├── scheduler.py                 # 任务调度策略（accept/reject 决策）
│    ├── inference.py                 # SGLang 接口
 (generate_until/loglikelihood)
│    └── config_loader.py             # 从 CONFIG_PATH 读取比赛配置
│
└── tests/
     ├── test_scorer.py
     ├── test_scheduler.py
     ├── test_inference.py             # 集成测试（需 SGLang 运行）
     └── test_client.py
```

Task 0：Clone SGLang 并本地安装

Files:

- Create: `sglang/`（子目录，clone 的源码）
- Modify: `contestant/setup.sh`（安装 sglang editable）
- Modify: `contestant/requirements.txt`（移除 `sglang>=0.3.0`，改为本地安装）
- ☐ **Step 1: Clone SGLang 源码**

```
cd /Users/hanzhuojun/WorkSpace/quant
git clone https://github.com/sgl-project/sglang.git
```

- ☐ **Step 2: Editable install（开发机）**

```
cd sglang
pip install -e "python/[all]"
# 验证
```

```
python -c "import sglang; print(sglang.__version__)"
# Expected: 0.x.x （打印版本号即为成功）
```

- ☐ **Step 3: 确认 SGLang 可以启动小模型**

```
# 用任意能跑的小模型验证启动正常（无需 Qwen）
python -m sglang.launch_server \
    --model-path facebook/opt-125m \
    --host 0.0.0.0 --port 30000 --tp-size 1 &
sleep 15
curl -s http://localhost:30000/health
# Expected: {"status":"ok"} 或类似
kill %1
```

- ☐ **Step 4: 更新 setup.sh 改为本地 editable install**

```
#!/usr/bin/env bash
# contestant/setup.sh
set -e

python3.12 -m venv .venv
source .venv/bin/activate
pip install -r contestant/requirements.txt
# sglang 从本地 clone 安装（提交时 sglang/ 目录一起打包）
pip install -e "sglang/python/[all]"
```

- ☐ **Step 5: 移除 requirements.txt 中的 sglang pip 行**

将 `contestant/requirements.txt` 中的 `sglang>=0.3.0` 这行删除（改为 setup.sh 本地安装）。

- ☐ **Step 6: Commit**

```
# sglang 源码本身不 commit（.gitignore 忽略），只 commit setup.sh 修改
echo "sglang/" >> .gitignore
git add .gitignore contestant/setup.sh contestant/requirements.txt
git commit -m "feat: use local SGLang clone for editable install"
```

Task 1：搭建项目骨架与配置

Files:

- Create: `mock_platform/config.py`
- Create: `contestant/config_loader.py`

- Create: `contestant/requirements.txt`
- ☐ **Step 1: 写 mock 配置（SLA 等级 + 采样参数，与官方文档完全一致）**

```
# mock_platform/config.py
SLA_LEVELS = {
    "Bronze":   {"ttft_avg": 10.0, "weight": 1.0},
    "Silver":   {"ttft_avg": 8.0,  "weight": 1.2},
    "Gold":     {"ttft_avg": 6.0,  "weight": 1.5},
    "Platinum": {"ttft_avg": 4.0,  "weight": 1.7},
    "Diamond":  {"ttft_avg": 2.0,  "weight": 2.0},
    "Stellar":  {"ttft_avg": 1.5,  "weight": 2.2},
    "Glorious": {"ttft_avg": 0.8,  "weight": 2.4},
    "Supreme":  {"ttft_avg": 0.5,  "weight": 2.5},
}

SAMPLING_PARAMS = {
    "Deterministic":  {"temperature": 0.0, "top_p": 1.0,  "top_k": 1,
"weight": 1.0},
    "Normal":         {"temperature": 0.1, "top_p": 0.9,  "top_k": 50,
"weight": 1.1},
    "HighEntropy":    {"temperature": 0.1, "top_p": 0.95, "top_k": 100,
"weight": 1.2},
    "ExtremePenalty": {"temperature": 0.1, "top_p": 0.9,  "top_k": 20,
"weight": 1.3},
}

TASK_TYPE_WEIGHTS = {
    "generate_until":       2.0,
    "loglikelihood":        1.0,
    "loglikelihood_rolling":1.0,
}

PENALTY_MULTIPLIER = 2.0   # 未完成扣分 = 2 × w_task × w_sla × w_sp
HARD_TIMEOUT_S = 600       # 超过此时间扣分
```

- ☐ **Step 2: 写 contestant 配置加载器**

```
# contestant/config_loader.py
import json, os
from dataclasses import dataclass

@dataclass
class ContestConfig:
    platform_url: str
    model_name: str
    model_path: str
    contestant_port: int
    duration_s: int
    sla_levels: dict
```

```
    sampling_params: dict

def load_config() -> ContestConfig:
    path = os.environ.get("CONFIG_PATH", "mock_platform/mock_config.json")
    with open(path) as f:
        d = json.load(f)
    return ContestConfig(
        platform_url=d["platform_url"],
        model_name=d["model_name"],
        model_path=d["model_path"],
        contestant_port=int(os.environ.get("CONTESTANT_PORT",
d.get("contestant_port", 9000))),
        duration_s=d["duration_s"],
        sla_levels=d["sla_levels"],
        sampling_params=d["sampling_params"],
    )
```

- ☐ **Step 3: 写 mock_config.json（热身阶段使用）**

```
// mock_platform/mock_config.json
{
  "platform_url": "http://localhost:8003",
  "model_name": "Qwen2.5-7B-Instruct",
  "model_path": "/path/to/Qwen2.5-7B-Instruct",
  "contestant_port": 9000,
  "duration_s": 3600,
  "sla_levels": {
    "Bronze":   {"ttft_avg": 10.0},
    "Silver":   {"ttft_avg": 8.0},
    "Gold":     {"ttft_avg": 6.0},
    "Platinum": {"ttft_avg": 4.0},
    "Diamond":  {"ttft_avg": 2.0},
    "Stellar":  {"ttft_avg": 1.5},
    "Glorious": {"ttft_avg": 0.8},
    "Supreme":  {"ttft_avg": 0.5}
  },
  "sampling_params": {
    "Deterministic":  {"temperature": 0.0, "top_p": 1.0,  "top_k": 1},
    "Normal":         {"temperature": 0.1, "top_p": 0.9,  "top_k": 50},
    "HighEntropy":    {"temperature": 0.1, "top_p": 0.95, "top_k": 100},
    "ExtremePenalty": {"temperature": 0.1, "top_p": 0.9,  "top_k": 20}
  }
}
```

- ☐ **Step 4: 写 requirements.txt**

```
fastapi>=0.111.0
uvicorn[standard]>=0.29.0
httpx>=0.27.0
```

```
sglang>=0.3.0
pydantic>=2.7.0
pytest>=8.2.0
pytest-asyncio>=0.23.0
```

- ☐ **Step 5: 验证结构**

```
cd /Users/hanzhuojun/WorkSpace/quant
python -c "from mock_platform.config import SLA_LEVELS;
print(SLA_LEVELS['Supreme'])"
# Expected: {'ttft_avg': 0.5, 'weight': 2.5}
CONFIG_PATH=mock_platform/mock_config.json python -c "
from contestant.config_loader import load_config
c = load_config(); print(c.platform_url)"
# Expected: http://localhost:8003
```

- ☐ **Step 6: Commit**

```
git init  # 若尚未初始化
git add mock_platform/config.py mock_platform/mock_config.json
contestant/config_loader.py contestant/requirements.txt
git commit -m "feat: project skeleton + config loader"
```

Task 2：Mock Platform 得分计算器

Files:

- Create: `mock_platform/scorer.py`
- Create: `tests/test_scorer.py`
- ☐ **Step 1: 写失败测试**

```
# tests/test_scorer.py
import sys; sys.path.insert(0, ".")
from mock_platform.scorer import calc_reward, calc_penalty

def test_reward_generate_until_bronze():
    r = calc_reward(
        task_type="generate_until",
        sla="Bronze",
        sampling_param="Deterministic",
        correctness=1.0
    )
    # w_task=2.0, w_sla=1.0, w_sp=1.0, C=1.0
    assert r == 2.0
```

```
def test_reward_loglikelihood_gold_normal():
    r = calc_reward(
        task_type="loglikelihood",
        sla="Gold",
        sampling_param="Normal",
        correctness=1.0
    )
    # w_task=1.0, w_sla=1.5, w_sp=1.0 (sp only applies to generate_until),
C=1.0
    assert r == 1.5

def test_reward_zero_correctness():
    r = calc_reward("generate_until", "Gold", "Deterministic",
correctness=0.0)
    assert r == 0.0

def test_penalty_generate_until_supreme():
    p = calc_penalty("generate_until", "Supreme", "ExtremePenalty")
    # 2 × 2.0 × 2.5 × 1.3 = 13.0
    assert p == pytest.approx(13.0)

def test_penalty_loglikelihood():
    p = calc_penalty("loglikelihood", "Bronze", "Normal")
    # 2 × 1.0 × 1.0 × 1.0 = 2.0 (sp not applied for loglikelihood)
    assert p == pytest.approx(2.0)
```

- ☐ **Step 2: 确认测试失败**

```
python -m pytest tests/test_scorer.py -v
# Expected: ImportError or FAILED
```

- ☐ **Step 3: 实现 scorer.py**

```
# mock_platform/scorer.py
import pytest
from mock_platform.config import TASK_TYPE_WEIGHTS, SLA_LEVELS,
SAMPLING_PARAMS, PENALTY_MULTIPLIER

def _w_sp(task_type: str, sampling_param: str) -> float:
    """采样参数权重：仅 generate_until 有效"""
    if task_type == "generate_until":
        return SAMPLING_PARAMS[sampling_param]["weight"]
    return 1.0

def calc_reward(task_type: str, sla: str, sampling_param: str,
correctness: float) -> float:
    """单题正常得分"""
    w_task = TASK_TYPE_WEIGHTS[task_type]
```

```
    w_sla  = SLA_LEVELS[sla]["weight"]
    w_sp   = _w_sp(task_type, sampling_param)
    return w_task * w_sla * w_sp * correctness

def calc_penalty(task_type: str, sla: str, sampling_param: str) -> float:
    """未完成（600s 超时）扣分"""
    w_task = TASK_TYPE_WEIGHTS[task_type]
    w_sla  = SLA_LEVELS[sla]["weight"]
    w_sp   = _w_sp(task_type, sampling_param)
    return PENALTY_MULTIPLIER * w_task * w_sla * w_sp
```

- ☐ **Step 4: 确认测试通过**

```
python -m pytest tests/test_scorer.py -v
# Expected: 5 passed
```

- ☐ **Step 5: Commit**

```
git add mock_platform/scorer.py tests/test_scorer.py
git commit -m "feat: scoring calculator with tests"
```

Task 3：Mock Platform 任务生成器

Files:

- Create: `mock_platform/task_generator.py`
- ☐ **Step 1: 实现任务生成器**

任务生成器的核心设计：

- 维护一个小型"题库"（prompt pool），故意复用 prompt，测试 KV Cache 命中
- loglikelihood 任务拆成多个 message（每个候选答案一个 message）
- generate_until 任务单 message
- 随机分配 SLA 等级（偏向中等：Gold/Platinum 居多）

```
# mock_platform/task_generator.py
import random, time, uuid
from dataclasses import dataclass, field
from typing import Optional

# 示例题库（热身阶段用假数据，正式预赛用真实 lm-eval 任务）
PROMPT_POOL = [
    {
        "prompt": "Question: What is 2+2?\nAnswer:",
        "type": "generate_until",
```

```
        "continuation": None,
        "until": ["\n"],
        "max_gen_toks": 16,
    },
    {
        "prompt": "The capital of France is",
        "type": "loglikelihood",
        "choices": ["Paris", "London", "Berlin", "Madrid"],
    },
    {
        "prompt": "Once upon a time in a land far away",
        "type": "loglikelihood_rolling",
        "continuation": None,
    },
    {
        "prompt": "Question: Natalia sold clips to 48 of her friends in
April, and then she sold half as many clips in May. How many clips did
Natalia sell altogether in April and May?\nAnswer:",
        "type": "generate_until",
        "continuation": None,
        "until": ["\n\n"],
        "max_gen_toks": 256,
    },
    {
        "prompt": "Which programming language is known for its use in data
science?",
        "type": "loglikelihood",
        "choices": ["Python", "Assembly", "COBOL", "Fortran"],
    },
]

SLA_DISTRIBUTION = ["Gold", "Gold", "Platinum", "Silver", "Bronze",
"Diamond"]

@dataclass
class TaskMessage:
    ID: int
    prompt: str
    eval_req_id: str
    eval_request_type: str
    eval_gen_kwargs: Optional[dict]
    eval_continuation: Optional[str]

@dataclass
class TaskOverview:
    task_id: int
    target_sla: str
    target_reward: float
    eval_task_name: str
    eval_request_type: str
    eval_sampling_param: str
    eval_timeout_s: int = 600

@dataclass
```

```python
class FullTask:
    overview: TaskOverview
    messages: list[TaskMessage]
    created_at: float = field(default_factory=time.time)
    reference_outputs: list = field(default_factory=list)  # for scoring

def _make_req_id() -> str:
    return f"w0_{uuid.uuid4().hex[:8]}"

def generate_task(task_id: int) -> FullTask:
    pool_item = random.choice(PROMPT_POOL)
    sla = random.choice(SLA_DISTRIBUTION)
    sp_name = random.choice(["Deterministic", "Normal"])

    if pool_item["type"] == "generate_until":
        msg = TaskMessage(
            ID=0,
            prompt=pool_item["prompt"],
            eval_req_id=_make_req_id(),
            eval_request_type="generate_until",
            eval_gen_kwargs={
                "until": pool_item["until"],
                "max_gen_toks": pool_item["max_gen_toks"],
                "temperature": 0.0 if sp_name == "Deterministic" else 0.1,
                "top_p": 1.0 if sp_name == "Deterministic" else 0.9,
                "top_k": 1 if sp_name == "Deterministic" else 50,
            },
            eval_continuation=None,
        )
        messages = [msg]
        ref_outputs = [None]  # 正式赛时由参考模型提供

    elif pool_item["type"] == "loglikelihood":
        messages = []
        ref_outputs = []
        for i, choice in enumerate(pool_item["choices"]):
            messages.append(TaskMessage(
                ID=i,
                prompt=pool_item["prompt"],
                eval_req_id=_make_req_id(),
                eval_request_type="loglikelihood",
                eval_gen_kwargs=None,
                eval_continuation=choice,
            ))
            ref_outputs.append(None)

    else:  # loglikelihood_rolling
        msg = TaskMessage(
            ID=0,
            prompt=pool_item["prompt"],
            eval_req_id=_make_req_id(),
            eval_request_type="loglikelihood_rolling",
            eval_gen_kwargs=None,
            eval_continuation=None,
```

```
        )
        messages = [msg]
        ref_outputs = [None]

    overview = TaskOverview(
        task_id=task_id,
        target_sla=sla,
        target_reward=2.5,  # mock: 后续由 scorer 精算
        eval_task_name=f"mock_task_{pool_item['type']}",
        eval_request_type=pool_item["type"],
        eval_sampling_param=sp_name,
    )
    return FullTask(overview=overview, messages=messages,
reference_outputs=ref_outputs)
```

- ☐ **Step 2: 快速验证生成逻辑**

```
python -c "
from mock_platform.task_generator import generate_task
t = generate_task(1)
print(t.overview.eval_request_type, len(t.messages),
t.overview.target_sla)
"
# Expected: loglikelihood 4 Gold   (或其他合法组合)
```

- ☐ **Step 3: Commit**

```
git add mock_platform/task_generator.py
git commit -m "feat: mock task generator with prompt pool"
```

Task 4：Mock Platform HTTP 服务器

Files:

- Create: `mock_platform/server.py`

- ☐ **Step 1: 实现 FastAPI 服务器**

```
# mock_platform/server.py
import asyncio, time
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional
from mock_platform.task_generator import generate_task, FullTask
from mock_platform.scorer import calc_reward, calc_penalty
from mock_platform.config import SLA_LEVELS
```

```python
app = FastAPI()

# --- 状态 ---
tokens: dict[str, str] = {}          # token -> team_name
task_counter = 1
available_tasks: list[FullTask] = [] # 待抢的任务队列
active_tasks: dict[int, dict] = {}   # task_id -> {task, token, ask_time}
completed_tasks: set[int] = set()
scores: dict[str, float] = {}        # token -> score

# --- 数据模型 ---
class RegisterReq(BaseModel):
    name: str
    token: str

class QueryReq(BaseModel):
    token: str

class AskReq(BaseModel):
    token: str
    task_id: int
    sla: str

class SubmitMsg(BaseModel):
    overview: dict
    messages: list[dict]

class SubmitReq(BaseModel):
    user: dict
    msg: SubmitMsg

# --- 后台任务：持续生成任务 ---
async def task_producer():
    global task_counter
    while True:
        if len(available_tasks) < 10:
            task = generate_task(task_counter)
            task_counter += 1
            available_tasks.append(task)
        await asyncio.sleep(0.5)

@app.on_event("startup")
async def startup():
    asyncio.create_task(task_producer())

# --- 接口 ---
@app.post("/register")
async def register(req: RegisterReq):
    tokens[req.token] = req.name
    scores.setdefault(req.token, 0.0)
    return {"status": "ok"}

@app.post("/query")
```

```
async def query(req: QueryReq):
    if req.token not in tokens:
        raise HTTPException(401, "unregistered token")
    if not available_tasks:
        raise HTTPException(404, "no tasks available")
    task = available_tasks[0]
    ov = task.overview
    return {
        "task_id": ov.task_id,
        "target_sla": ov.target_sla,
        "target_reward": ov.target_reward,
        "eval_task_name": ov.eval_task_name,
        "eval_request_type": ov.eval_request_type,
        "eval_sampling_param": ov.eval_sampling_param,
        "eval_timeout_s": ov.eval_timeout_s,
    }

@app.post("/ask")
async def ask(req: AskReq):
    if req.token not in tokens:
        raise HTTPException(401)
    # 找到对应任务
    task = next((t for t in available_tasks if t.overview.task_id ==
req.task_id), None)
    if task is None:
        return {"status": "closed"}
    # 初赛：SLA 必须与 target_sla 完全一致
    if req.sla != task.overview.target_sla:
        return {"status": "rejected", "reason": "SLA must match target"}
    available_tasks.remove(task)
    active_tasks[req.task_id] = {
        "task": task, "token": req.token, "ask_time": time.time()
    }
    return {
        "status": "accepted",
        "task": {
            "overview": vars(task.overview),
            "messages": [vars(m) for m in task.messages],
        }
    }

@app.post("/submit")
async def submit(req: SubmitReq):
    token = req.user.get("token")
    if token not in tokens:
        raise HTTPException(401)
    task_id = req.msg.overview.get("task_id")
    if task_id in completed_tasks:
        return {"status": "ok"}  # 幂等
    rec = active_tasks.get(task_id)
    if rec is None:
        raise HTTPException(404, "task not found")

    elapsed = time.time() - rec["ask_time"]
```

```
    task: FullTask = rec["task"]
    ov = task.overview
    sla_ttft = SLA_LEVELS[ov.target_sla]["ttft_avg"]

    # Mock 正确性：热身阶段假设 correctness=1.0（无参考模型），
    # 正式接入参考模型后此处替换
    correctness = 1.0

    if elapsed <= sla_ttft:
        reward = calc_reward(ov.eval_request_type, ov.target_sla,
ov.eval_sampling_param, correctness)
        scores[token] = scores.get(token, 0.0) + reward
    elif elapsed <= 600:
        pass  # SLA 超时但 600s 内：不得分不扣分
    else:
        penalty = calc_penalty(ov.eval_request_type, ov.target_sla,
ov.eval_sampling_param)
        scores[token] = scores.get(token, 0.0) - penalty

    completed_tasks.add(task_id)
    active_tasks.pop(task_id, None)
    return {"status": "ok"}

@app.get("/scores")
async def get_scores():
    return {tokens.get(t, t): s for t, s in scores.items()}

# 启动命令：
# cd /Users/hanzhuojun/WorkSpace/quant
# python -m uvicorn mock_platform.server:app --host 0.0.0.0 --port 8003
```

- ☐ **Step 2: 启动并手动验证接口**

```
# 终端 1
python -m uvicorn mock_platform.server:app --host 0.0.0.0 --port 8003

# 终端 2
curl -s -X POST http://localhost:8003/register \
  -H "Content-Type: application/json" \
  -d '{"name":"test_team","token":"abc123"}' | python -m json.tool
# Expected: {"status": "ok"}

curl -s -X POST http://localhost:8003/query \
  -H "Content-Type: application/json" \
  -d '{"token":"abc123"}' | python -m json.tool
# Expected: task overview JSON with task_id, target_sla, etc.
```

- ☐ **Step 3: Commit**

```
git add mock_platform/server.py
git commit -m "feat: mock platform HTTP server with scoring"
```

Task 5：SGLang 推理后端接口

Files:

- Create: `contestant/inference.py`
- Create: `tests/test_inference.py`

> **前置条件：** SGLang 服务已启动。热身阶段命令：
>
> ```
> python -m sglang.launch_server \
>  --model-path /path/to/Qwen2.5-7B-Instruct \
>  --host 0.0.0.0 --port 30000 --tp-size 1
> ```

- ☐ **Step 1: 实现三种推理模式**

```
# contestant/inference.py
import httpx
from typing import Optional

class SGLangClient:
    """封装 SGLang 原生 HTTP API，支持三种评测请求类型。"""

    def __init__(self, base_url: str, model_name: str, timeout: float =
120.0):
        self.base_url = base_url.rstrip("/")
        self.model_name = model_name
        self.timeout = timeout
        self._client = httpx.Client(timeout=timeout)

    def generate_until(self, prompt: str, gen_kwargs: dict) -> str:
        """generate_until: 生成文本，遇到 until 停止词则截断。"""
        stop = gen_kwargs.get("until", [])
        resp = self._client.post(
            f"{self.base_url}/v1/completions",
            json={
                "model": self.model_name,
                "prompt": prompt,
                "max_tokens": gen_kwargs.get("max_gen_toks", 256),
                "temperature": gen_kwargs.get("temperature", 0.0),
                "top_p": gen_kwargs.get("top_p", 1.0),
                "top_k": gen_kwargs.get("top_k", 1),
                "stop": stop if stop else None,
            },
        )
```

```
        resp.raise_for_status()
        return resp.json()["choices"][0]["text"]

    def loglikelihood(self, prompt: str, continuation: str) -> float:
        """loglikelihood: 返回 log P(continuation | prompt)。

        使用 SGLang 原生 /generate 端点获取 input_token_logprobs。
        continuation 对应的 token logprobs 之和即为所求。
        """
        full_text = prompt + continuation
        resp = self._client.post(
            f"{self.base_url}/generate",
            json={
                "text": full_text,
                "sampling_params": {"max_new_tokens": 1, "temperature":
0.0},
                "return_logprob": True,
                "input_token_logprobs": True,
                "return_text_in_logprobs": True,
            },
        )
        resp.raise_for_status()
        data = resp.json()
        # input_token_logprobs: list of (logprob, token_id, token_text)
        token_logprobs = data.get("input_token_logprobs", [])
        # 用分词估算 continuation 占多少 token（粗估：字符数/4）
        # 准确做法：用 tokenizer 计算
        continuation_token_count = max(1, len(continuation) // 4)
        return sum(lp[0] for lp in token_logprobs[-
continuation_token_count:])

    def loglikelihood_rolling(self, prompt: str) -> float:
        """loglikelihood_rolling: 计算整段 prompt 的 total log-
likelihood。"""
        resp = self._client.post(
            f"{self.base_url}/generate",
            json={
                "text": prompt,
                "sampling_params": {"max_new_tokens": 1, "temperature":
0.0},
                "return_logprob": True,
                "input_token_logprobs": True,
            },
        )
        resp.raise_for_status()
        data = resp.json()
        token_logprobs = data.get("input_token_logprobs", [])
        return sum(lp[0] for lp in token_logprobs[1:])  # 跳过第一个
token（无前文）

    def close(self):
        self._client.close()
```

> **注意：** `loglikelihood` 中的 continuation token count 需用 tokenizer 精确计算。Task 6 会修正此处。

- ☐ **Step 2: 写集成测试（需 SGLang 在 30000 端口运行）**

```
# tests/test_inference.py
import pytest
from contestant.inference import SGLangClient

SGLANG_URL = "http://localhost:30000"
MODEL = "Qwen2.5-7B-Instruct"

@pytest.fixture
def client():
    c = SGLangClient(SGLANG_URL, MODEL)
    yield c
    c.close()

@pytest.mark.integration
def test_generate_until(client):
    result = client.generate_until(
        prompt="The capital of France is",
        gen_kwargs={"until": ["\n"], "max_gen_toks": 20, "temperature":
0.0, "top_p": 1.0, "top_k": 1}
    )
    assert isinstance(result, str)
    assert len(result) > 0

@pytest.mark.integration
def test_loglikelihood_ordering(client):
    """正确答案的 logprob 应高于错误答案"""
    prompt = "The capital of France is"
    lp_paris  = client.loglikelihood(prompt, " Paris")
    lp_london = client.loglikelihood(prompt, " London")
    assert lp_paris > lp_london

@pytest.mark.integration
def test_loglikelihood_rolling_is_negative(client):
    lp = client.loglikelihood_rolling("Once upon a time in a land far
away")
    assert lp < 0  # log-prob 必为负数
```

- ☐ **Step 3: 运行集成测试（需 SGLang 启动）**

```
python -m pytest tests/test_inference.py -m integration -v
# Expected: 3 passed   (若 SGLang 未启动则 skip/fail, 正常)
```

- ☐ **Step 4: Commit**

```
git add contestant/inference.py tests/test_inference.py
git commit -m "feat: SGLang inference client for all 3 eval types"
```

Task 6：精确 Loglikelihood（Tokenizer 修正）

Files:

- Modify: `contestant/inference.py`

> **问题：** Task 5 中用字符数估算 continuation token count，不准确。正确做法是用 tokenizer 计算。

- ☐ **Step 1: 更新 SGLangClient，注入 tokenizer**

```
# contestant/inference.py — 在 __init__ 中添加:
from transformers import AutoTokenizer

class SGLangClient:
    def __init__(self, base_url: str, model_name: str, model_path: str,
timeout: float = 120.0):
        self.base_url = base_url.rstrip("/")
        self.model_name = model_name
        self.timeout = timeout
        self._client = httpx.Client(timeout=timeout)
        # tokenizer 用于精确计算 continuation token 数量
        self._tokenizer = AutoTokenizer.from_pretrained(model_path,
use_fast=True)

    def loglikelihood(self, prompt: str, continuation: str) -> float:
        full_text = prompt + continuation
        # 精确计算 continuation 的 token 数
        prompt_ids = self._tokenizer.encode(prompt,
add_special_tokens=False)
        full_ids   = self._tokenizer.encode(full_text,
add_special_tokens=False)
        continuation_token_count = len(full_ids) - len(prompt_ids)

        resp = self._client.post(
            f"{self.base_url}/generate",
            json={
                "text": full_text,
                "sampling_params": {"max_new_tokens": 1, "temperature":
0.0},
                "return_logprob": True,
                "input_token_logprobs": True,
            },
        )
        resp.raise_for_status()
        data = resp.json()
        token_logprobs = data.get("input_token_logprobs", [])
```

```
        return sum(lp[0] for lp in token_logprobs[-
continuation_token_count:])
```

- ☐ **Step 2: 更新 requirements.txt 加 transformers**

在 `contestant/requirements.txt` 末尾添加：

```
transformers>=4.40.0
```

- ☐ **Step 3: 重跑集成测试确认精度改善**

```
python -m pytest tests/test_inference.py::test_loglikelihood_ordering -m
integration -v
# Expected: PASSED
```

- ☐ **Step 4: Commit**

```
git add contestant/inference.py contestant/requirements.txt
git commit -m "fix: use tokenizer for precise continuation token count in
loglikelihood"
```

Task 7：平台 API 客户端

Files:

- Create: `contestant/client.py`

- ☐ **Step 1: 实现客户端**

```
# contestant/client.py
import httpx
from typing import Optional

class PlatformClient:
    """与评测平台（真实或 mock）交互的 HTTP 客户端。"""

    def __init__(self, platform_url: str, token: str, team_name: str,
timeout: float = 30.0):
        self.platform_url = platform_url.rstrip("/")
        self.token = token
        self.team_name = team_name
        self._client = httpx.Client(timeout=timeout)

    def register(self) -> bool:
```

```
        resp = self._client.post(
            f"{self.platform_url}/register",
            json={"name": self.team_name, "token": self.token},
        )
        resp.raise_for_status()
        return resp.json().get("status") == "ok"

    def query(self) -> Optional[dict]:
        """拉取一道任务概要。无任务时返回 None。"""
        resp = self._client.post(
            f"{self.platform_url}/query",
            json={"token": self.token},
        )
        if resp.status_code == 404:
            return None
        resp.raise_for_status()
        return resp.json()

    def ask(self, task_id: int, sla: str) -> Optional[dict]:
        """接受任务，返回完整任务数据；若被拒绝/已关闭返回 None。"""
        resp = self._client.post(
            f"{self.platform_url}/ask",
            json={"token": self.token, "task_id": task_id, "sla": sla},
        )
        resp.raise_for_status()
        data = resp.json()
        if data.get("status") == "accepted":
            return data["task"]
        return None  # rejected or closed

    def submit(self, task_overview: dict, messages: list[dict]) -> bool:
        resp = self._client.post(
            f"{self.platform_url}/submit",
            json={
                "user": {"name": self.team_name, "token": self.token},
                "msg": {"overview": task_overview, "messages": messages},
            },
        )
        resp.raise_for_status()
        return resp.json().get("status") == "ok"

    def close(self):
        self._client.close()
```

- ☐ **Step 2: 写客户端单元测试（mock HTTP）**

```
# tests/test_client.py
import pytest
from unittest.mock import patch, MagicMock
from contestant.client import PlatformClient

@pytest.fixture
```

```
def client():
    return PlatformClient("http://localhost:8003", "tok123", "team_a")

def test_register_success(client):
    mock_resp = MagicMock()
    mock_resp.json.return_value = {"status": "ok"}
    mock_resp.status_code = 200
    with patch.object(client._client, "post", return_value=mock_resp):
        assert client.register() is True

def test_query_returns_none_on_404(client):
    mock_resp = MagicMock()
    mock_resp.status_code = 404
    with patch.object(client._client, "post", return_value=mock_resp):
        result = client.query()
    assert result is None

def test_ask_rejected_returns_none(client):
    mock_resp = MagicMock()
    mock_resp.json.return_value = {"status": "rejected", "reason": "SLA
must match target"}
    mock_resp.status_code = 200
    with patch.object(client._client, "post", return_value=mock_resp):
        result = client.ask(42, "Gold")
    assert result is None
```

- ☐ **Step 3: 运行单元测试**

```
python -m pytest tests/test_client.py -v
# Expected: 3 passed
```

- ☐ **Step 4: Commit**

```
git add contestant/client.py tests/test_client.py
git commit -m "feat: platform API client with unit tests"
```

Task 8：任务调度策略（Scheduler）

Files:

- Create: `contestant/scheduler.py`
- Create: `tests/test_scheduler.py`

调度策略的核心：在收到任务概要（/query 响应）后，决定是否调用 /ask 接受该任务。 初赛时 SLA 必须完全匹配，所以决策的关键是**当前系统负载能否满足该 SLA 的 TTFT 要求**。

- ☐ **Step 1: 写失败测试**

```
# tests/test_scheduler.py
from contestant.scheduler import Scheduler

def test_accept_task_when_idle():
    s = Scheduler(max_concurrent=4)
    overview = {
        "task_id": 1,
        "target_sla": "Gold",          # ttft_avg=6.0s
        "eval_request_type": "generate_until",
        "eval_sampling_param": "Deterministic",
    }
    assert s.should_accept(overview) is True

def test_reject_when_overloaded():
    s = Scheduler(max_concurrent=2)
    s.mark_active(1)
    s.mark_active(2)
    overview = {"task_id": 3, "target_sla": "Supreme",
"eval_request_type": "generate_until", "eval_sampling_param":
"Deterministic"}
    # Supreme ttft=0.5s, 当前已满负载, 拒绝
    assert s.should_accept(overview) is False

def test_always_accept_low_sla_when_idle():
    s = Scheduler(max_concurrent=4)
    overview = {"task_id": 4, "target_sla": "Bronze", "eval_request_type":
"loglikelihood", "eval_sampling_param": "Normal"}
    assert s.should_accept(overview) is True

def test_mark_complete_frees_slot():
    s = Scheduler(max_concurrent=1)
    s.mark_active(1)
    s.mark_complete(1)
    overview = {"task_id": 2, "target_sla": "Bronze", "eval_request_type":
"loglikelihood", "eval_sampling_param": "Normal"}
    assert s.should_accept(overview) is True
```

- ☐ **Step 2: 确认测试失败**

```
python -m pytest tests/test_scheduler.py -v
# Expected: ImportError or FAILED
```

- ☐ **Step 3: 实现 Scheduler**

```
# contestant/scheduler.py
from mock_platform.config import SLA_LEVELS

# SLA 时限越短越需要谨慎接受（需要更低负载）
```

```python
SLA_LOAD_THRESHOLDS = {
    "Bronze":   1.0,   # 负载比例 <= 1.0 时接受（总是接受）
    "Silver":   0.9,
    "Gold":     0.8,
    "Platinum": 0.7,
    "Diamond":  0.6,
    "Stellar":  0.5,
    "Glorious": 0.4,
    "Supreme":  0.3,
}

class Scheduler:
    def __init__(self, max_concurrent: int = 8):
        self.max_concurrent = max_concurrent
        self._active: set[int] = set()

    @property
    def load_ratio(self) -> float:
        return len(self._active) / self.max_concurrent

    def should_accept(self, overview: dict) -> bool:
        sla = overview.get("target_sla", "Bronze")
        threshold = SLA_LOAD_THRESHOLDS.get(sla, 1.0)
        return self.load_ratio < threshold

    def mark_active(self, task_id: int):
        self._active.add(task_id)

    def mark_complete(self, task_id: int):
        self._active.discard(task_id)
```

- ☐ **Step 4: 确认测试通过**

```bash
python -m pytest tests/test_scheduler.py -v
# Expected: 4 passed
```

- ☐ **Step 5: Commit**

```bash
git add contestant/scheduler.py tests/test_scheduler.py
git commit -m "feat: SLA-aware task scheduler with load threshold"
```

Task 9：主控循环与提交脚本

Files:

- Create: `contestant/main.py`

- Create: `contestant/run.sh`
- Create: `contestant/setup.sh`
- ☐ **Step 1: 实现主循环**

```
# contestant/main.py
import time, os, sys, logging
from contestant.config_loader import load_config
from contestant.client import PlatformClient
from contestant.inference import SGLangClient
from contestant.scheduler import Scheduler

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s")
log = logging.getLogger(__name__)

TEAM_NAME = os.environ.get("TEAM_NAME", "team_alpha")
TOKEN     = os.environ.get("TEAM_TOKEN", "secret_token")
SGLANG_URL = os.environ.get("SGLANG_URL", "http://localhost:30000")

def process_task(task_data: dict, inference: SGLangClient) -> list[dict]:
    """对 task 中每条 message 执行推理，返回填充后的 messages 列表。"""
    messages = task_data["messages"]
    result_messages = []
    for msg in messages:
        req_type = msg["eval_request_type"]
        m = dict(msg)  # 拷贝，避免修改原数据
        if req_type == "generate_until":
            m["response"] = inference.generate_until(msg["prompt"],
msg["eval_gen_kwargs"])
            m["accuracy"] = None
        elif req_type == "loglikelihood":
            m["accuracy"] = inference.loglikelihood(msg["prompt"],
msg["eval_continuation"])
            m["response"] = None
        elif req_type == "loglikelihood_rolling":
            m["accuracy"] = inference.loglikelihood_rolling(msg["prompt"])
            m["response"] = None
        result_messages.append(m)
    return result_messages

def main():
    cfg = load_config()
    log.info(f"Config loaded: platform={cfg.platform_url}, model=
{cfg.model_name}")

    platform = PlatformClient(cfg.platform_url, TOKEN, TEAM_NAME)
    inference = SGLangClient(SGLANG_URL, cfg.model_name, cfg.model_path)
    scheduler = Scheduler(max_concurrent=8)

    platform.register()
    log.info("Registered with platform")
```

```
    start = time.time()
    while time.time() - start < cfg.duration_s:
        overview = platform.query()
        if overview is None:
            time.sleep(0.1)
            continue

        if not scheduler.should_accept(overview):
            time.sleep(0.05)
            continue

        task_id = overview["task_id"]
        task_data = platform.ask(task_id, overview["target_sla"])
        if task_data is None:
            continue  # rejected or closed

        scheduler.mark_active(task_id)
        log.info(f"Accepted task {task_id} SLA={overview['target_sla']}
type={overview['eval_request_type']}")

        try:
            result_messages = process_task(task_data, inference)
            ok = platform.submit(task_data["overview"], result_messages)
            log.info(f"Submitted task {task_id}: {'ok' if ok else
'fail'}")
        except Exception as e:
            log.error(f"Task {task_id} failed: {e}")
        finally:
            scheduler.mark_complete(task_id)

    inference.close()
    platform.close()
    log.info("Done")

if __name__ == "__main__":
    main()
```

- ☐ **Step 2: 写 run.sh（正式提交必须）**

```
#!/usr/bin/env bash
# contestant/run.sh
set -e

# 激活虚拟环境（若 setup.sh 创建了的话）
if [ -f ".venv/bin/activate" ]; then
    source .venv/bin/activate
fi

# 启动 SGLang 后端（后台运行）
python -m sglang.launch_server \
    --model-path "${MODEL_PATH}" \
```

```
    --host 0.0.0.0 --port 30000 \
    --tp-size 1 &
SGLANG_PID=$!

# 等待 SGLang 就绪（最多 55s，run.sh 总时限 60s）
for i in $(seq 1 55); do
    if curl -s http://localhost:30000/health > /dev/null 2>&1; then
        echo "SGLang ready after ${i}s"
        break
    fi
    sleep 1
done

# 启动选手客户端
SGLANG_URL=http://localhost:30000 \
    python -m contestant.main

# 清理
kill $SGLANG_PID 2>/dev/null || true
```

- ☐ **Step 3: 写 setup.sh**

```
#!/usr/bin/env bash
# contestant/setup.sh
set -e

python3.12 -m venv .venv
source .venv/bin/activate
pip install -r contestant/requirements.txt
```

- ☐ **Step 4: 赋予执行权限**

```
chmod +x contestant/run.sh contestant/setup.sh
```

- ☐ **Step 5: Commit**

```
git add contestant/main.py contestant/run.sh contestant/setup.sh
git commit -m "feat: main control loop + submission scripts"
```

Task 10：端到端热身模拟测试

Files:

- 不新增文件，验证整体流程

- ☐ **Step 1: 启动 SGLang（终端 1）**

```
python -m sglang.launch_server \
    --model-path /path/to/Qwen2.5-7B-Instruct \
    --host 0.0.0.0 --port 30000 --tp-size 1
# 等待出现 "Server is ready"
```

- ☐ **Step 2: 启动 mock 平台（终端 2）**

```
CONFIG_PATH=mock_platform/mock_config.json \
    python -m uvicorn mock_platform.server:app --host 0.0.0.0 --port 8003
```

- ☐ **Step 3: 运行选手客户端（终端 3，限时 60s 测试）**

```
CONFIG_PATH=mock_platform/mock_config.json \
    TEAM_TOKEN=mytoken \
    SGLANG_URL=http://localhost:30000 \
    python -c "
import os; os.environ['DURATION_OVERRIDE']='60'
from contestant.main import main
main()
"
# Expected: 看到 Accepted task / Submitted task 日志，无 ERROR
```

- ☐ **Step 4: 查看得分**

```
curl -s http://localhost:8003/scores | python -m json.tool
# Expected: {"team_alpha": <正数>}
```

- ☐ **Step 5: 确认 KV Cache 命中（SGLang 日志）**

```
# 在 SGLang 终端观察日志中 "cache hit rate" 字样
# 重复 prompt 多次后命中率应 > 0
```

- ☐ **Step 6: Commit**

```
git add .
git commit -m "feat: end-to-end warmup simulation working"
```

关键优化说明（已内置，无需额外代码）

1. Prefix Sharing（已内置于 SGLang）

SGLang 默认启用 RadixAttention，对 loglikelihood 任务中同一 prompt 的多个 continuation，KV Cache 自动命中——无需额外代码。

2. 请求去重

Mock 平台会复用 prompt（题库抽取机制），SGLang 的 RadixAttention 自动处理，命中率随时间提高。

3. SLA 感知调度（Task 8 的 Scheduler）

通过负载阈值矩阵，在高 SLA 任务（Supreme/Glorious）只在低负载时接受，避免超时扣分。

4. 后续可扩展优化（正式预赛前）

- **多线程并发**：将 `main.py` 的主循环改为 `asyncio` + `httpx.AsyncClient`，并发处理多任务
- **动态 max_concurrent**：根据实时 TTFT 反馈动态调整 Scheduler 的并发上限
- **Bid 策略（决赛）**：根据信誉分动态调整接单频率

验证清单

- [ ] `python -m pytest tests/ -v --ignore=tests/test_inference.py` 全部通过（无需 GPU）
- [ ] Mock 平台启动后，手动 curl 4 个接口全部正常响应
- [ ] 选手客户端 60s 模拟运行无崩溃，最终得分为正数
- [ ] SGLang 日志显示 cache hit rate > 0（说明 prefix sharing 生效）
- [ ] `contestant/run.sh` 在干净环境下可独立启动（正式提交前验证）